

Carpeta Ciudadana

Software Architecture Specification

1.0

Septiembre de 2026

Santiago Alexander Cardenas Laverde, Jhon
Emmanuel correa correa, Juan Miguel Castro
Martinez
Lead Software Engineer

Prepared for
Arquitectura de Software Moderna — Sistemas Distribuidos
Instructor: Danny Andrés Salcedo, MSc. Ingeniería de Software.
Septiembre 2026

Tabla de contenido

1. Introducción	3
1.1 Propósito y alcance de la entrega	3
1.2 Relación con la entrega 1	3
2. Microservicios y responsabilidades	4
2.1 Criterio de descomposición	4
2.2 Catálogo de microservicios	4
3. Casos de uso	6
3.1 Diagrama de casos de uso	6
3.2 Especificación de los casos de uso	7
4. Vista de componentes	12
4.1 Componentes lógicos	12
4.2 Componentes técnicos	13
5. Diagramas de secuencia	15
5.1 CU-01 Registro de un ciudadano	16
5.2 CU-02 Autenticación del ciudadano	18
5.3 CU-03 y CU-04 Carga y autenticación de documento	19
5.4 CU-09 Traslado a otro operador	21
6. Contrato de interoperabilidad entre operadores	22
6.1 Acuerdo adoptado	22
6.2 Brechas identificadas en el acuerdo	23
7. Diagrama de despliegue	24
7.1 Diagrama	24
7.2 Protocolos y formatos de mensajería	26
8. Decisiones de arquitectura	27
8.1 AD-01 Estilo arquitectural	27
8.2 AD-02 Modelo de despliegue	28
8.3 AD-03 Plataforma de ejecución	29
8.4 AD-04 Lenguaje y marco de trabajo	30
8.5 AD-05 Persistencia y almacenamiento documental	31
8.6 AD-06 Comunicación entre servicios	32
8.7 AD-07 Gestión de identidad	33
8.8 AD-08 Transferencia entre operadores	34
9. Alcance de la implementación	35
10. Trazabilidad con la entrega 1	35

1. Introducción

1.1 Propósito y alcance de la entrega

Este documento define la arquitectura de la solución de Operador de Carpeta Ciudadana. Mientras la entrega 1 estableció qué debe hacer el sistema y con qué calidad, esta entrega establece cómo se construye: qué microservicios lo componen, qué responsabilidad tiene cada uno, cómo se comunican, sobre qué tecnologías se ejecutan, dónde se despliegan y por qué se tomó cada decisión.

El alcance de implementación de la entrega comprende cuatro casos de uso: el registro de un ciudadano en la carpeta, la autenticación del ciudadano ante el operador, la carga de documentos y la autenticación de documentos a través de GovCarpeta. El resto de la solución queda especificado arquitecturalmente pero se construye en entregas posteriores.

1.2 Relación con la entrega 1

Todo lo que aquí se define es trazable a la especificación anterior. Los casos de uso derivan de los 44 requerimientos funcionales; los límites de los microservicios reproducen las ocho capacidades de negocio identificadas; y cada decisión de arquitectura referencia el requerimiento no funcional que la motiva, tomado del mapeo de QoS del capítulo 5.

Conviene recordar el argumento central de la entrega anterior, porque es el que justifica todo lo que sigue. Los requerimientos funcionales de Carpeta Ciudadana podrían satisfacerse con un monolito bien construido. Lo que descarta esa opción son los requerimientos no funcionales: disponibilidad mensual del 99,9 % que sube al 99,95 % en la consulta documental, capacidad de convertir recursos adicionales en throughput, integridad documental verificable y despliegue acotado al servicio modificado. La arquitectura que se define aquí es la respuesta a esos números, no a la lista de funcionalidades.

Además, esta entrega cierra el riesgo que la entrega 1 dejó abierto en su capítulo 8.4 y calificó como el de mayor impacto del proyecto: la transferencia directa entre operadores solo podía demostrarse si los equipos acordaban un contrato común, y ese contrato no existía. El acuerdo ya se alcanzó y se documenta en el capítulo 6, junto con seis brechas que todavía no resuelve.

2. Microservicios y responsabilidades

2.1 Criterio de descomposición

Los límites entre servicios se trazaron por capacidad de negocio, no por capa técnica. La diferencia no es cosmética: un corte por capa produce un servicio de controladores, uno de lógica y uno de acceso a datos, y cualquier cambio funcional obliga a tocar los tres y desplegarlos juntos. Eso es un monolito distribuido, que reúne el acoplamiento del monolito con la complejidad operativa de los microservicios.

El criterio aplicado es el principio de responsabilidad única llevado a escala de sistema: cada servicio agrupa lo que cambia por la misma razón. La columna de razón de cambio de la tabla siguiente es la prueba de ese criterio. Si dos servicios comparten razón de cambio, sobra uno; si un servicio tiene dos, hay que partirlo.

2.2 Catálogo de microservicios

Cada servicio posee su propio almacén de datos y ninguno consulta la base de otro. Esa regla es la que hace real la independencia de despliegue: sin ella, un cambio de esquema se propaga a servicios que no lo esperaban.

Microservicio	Responsabilidad	Razón de cambio	Datos propios	RF que cubre
ms-ciudadano	Registro, afiliación única, generación e inmutabilidad de la cuenta de correo, ciclo de vida y traslado del titular.	El estado de afiliación del ciudadano cambia.	ciudadano_db (PostgreSQL)	RF-01 a RF-11
ms-documentos	Metadatos, cuotas, estados del documento, emisión de URL firmadas y certificación.	Cambian las reglas sobre los documentos o sus metadatos.	documento_db + almacén de objetos	RF-12 a RF-22
ms-solicitudes	Peticiones entre partes, consentimiento del titular, armado de paquetes y enrutamiento del envío.	Cambian las reglas de autorización o de enrutamiento.	solicitud_db (PostgreSQL)	RF-23 a RF-30
ms-notificaciones	Composición y entrega de avisos por correo, SMS y bandeja del portal.	Cambia un canal o una plantilla de notificación.	notif_db (PostgreSQL)	RF-31 a RF-33
ms-interoperabilidad	Capa anticorrupción frente a GovCarpeta, saga de transferencia y endpoints de recepción entre operadores.	Cambia el contrato del centralizador o el de los operadores pares.	interop_db (PostgreSQL)	RF-34 a RF-36, RF-09, RF-10, RF-28, RF-29
ms-premium	Casos PQRS, solicitud de soportes a clientes y medición del consumo facturable.	Cambian las reglas comerciales del servicio premium.	premium_db (PostgreSQL)	RF-40 a RF-43
Keycloak (producto)	Autenticación, segundo factor, emisión y rotación de tokens, roles.	Cambian las políticas de identidad.	Almacén propio de Keycloak	RF-37, RF-38

Dos requerimientos no aparecen asignados a un único servicio porque no lo están. El RF-39, registro de operaciones relevantes, es transversal: cada servicio emite sus propios eventos de auditoría hacia una bitácora común. El RF-44, disponibilidad de metadatos para análisis, se atiende con un consumidor de eventos que se construye en una fase posterior y que no accede a las bases de los servicios de dominio, sino a la proyección que estos publican.

3. Casos de uso

3.1 Diagrama de casos de uso

El diagrama identifica doce casos de uso y los actores que los inician. Los cuatro sombreados en verde son los que se implementan en esta entrega.

Diagrama de casos de uso — Operador de Carpeta Ciudadana

Los casos de uso en verde son los que se implementan en la entrega 2

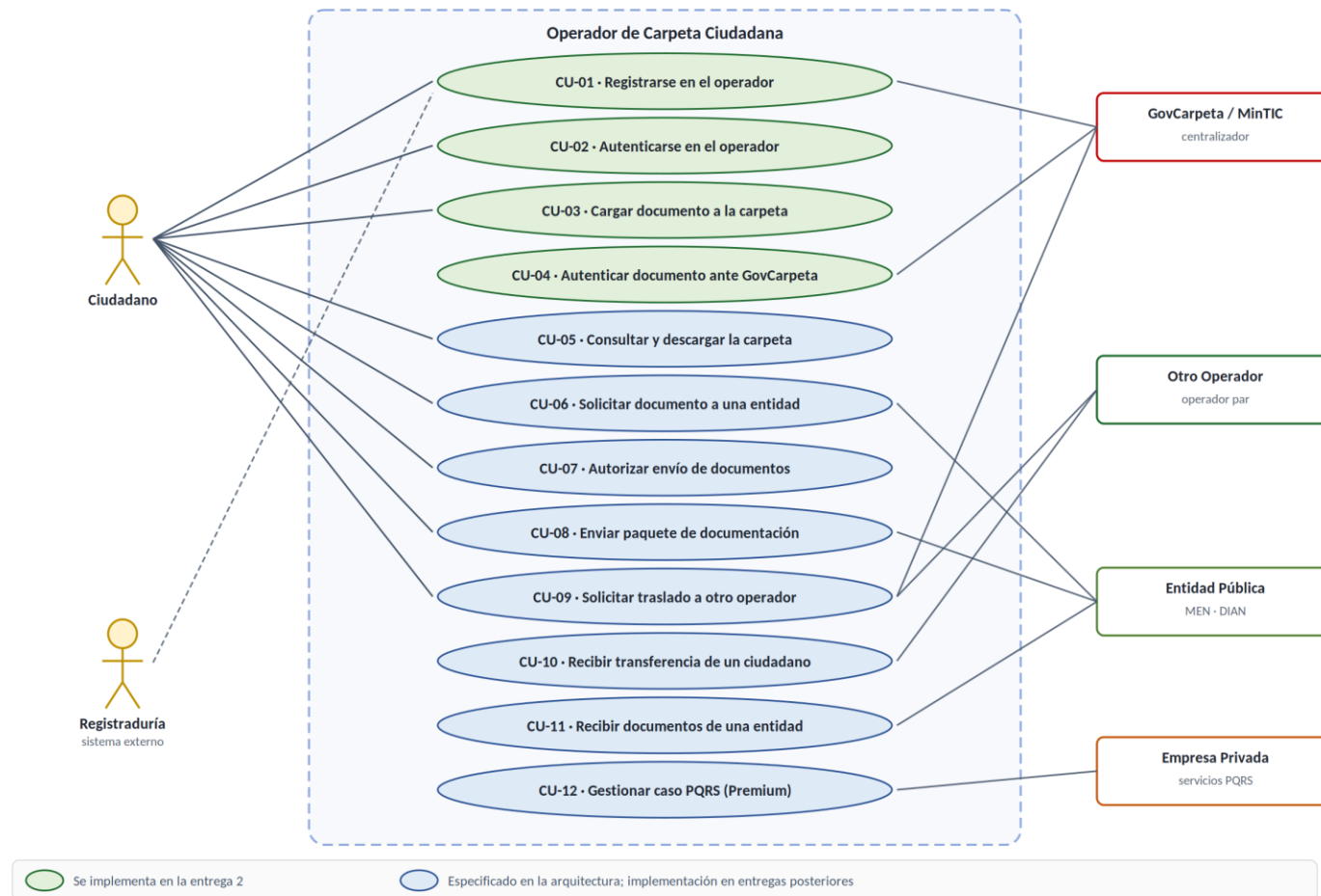


Figura 1. Diagrama de casos de uso del Operador de Carpeta Ciudadana.

3.2 Especificación de los casos de uso

Se especifican los cuatro casos de uso que se implementan, más el traslado entre operadores por ser el que concentra el mayor riesgo funcional de la solución.

CU-01 · Registrarse en el operador

Identificador	CU-01
Nombre	Registrarse en el operador
Actor principal	Ciudadano
Actores secundarios	Registraduría Nacional, GovCarpeta (MinTIC)
Descripción	Un ciudadano crea su carpeta en el operador. El sistema valida su identidad, verifica que no esté afiliado a otro operador, genera su cuenta de correo institucional y notifica la afiliación al centralizador.
Disparador	El ciudadano solicita crear su carpeta desde el portal.
Precondiciones	El ciudadano no tiene carpeta activa en este operador y dispone de un documento de identidad válido.
Postcondiciones	Existe una carpeta activa, el ciudadano queda afiliado a este operador en el directorio del centralizador y tiene asignada una cuenta @carpetacolombia.co inmutable.
Flujo principal	1. El ciudadano ingresa sus datos personales y su número de documento. 2. El sistema consulta a la Registraduría y valida la identidad declarada. 3. El sistema consulta al centralizador si el ciudadano está disponible. 4. El centralizador responde que no tiene operador asignado. 5. El sistema crea la carpeta y genera la cuenta de correo institucional. 6. El sistema notifica la afiliación al centralizador. 7. El sistema publica el evento de registro y confirma al ciudadano.
Flujos alternos y de excepción	3a. El ciudadano ya está afiliado a otro operador: el sistema rechaza el registro e informa que la vía disponible es solicitar un traslado. 2a. La Registraduría no confirma la identidad: el sistema rechaza el registro y registra el intento. 6a. El centralizador no responde: la notificación se encola y se reintenta con retroceso exponencial; la carpeta queda en estado pendiente de confirmación y no admite operaciones de intercambio hasta confirmarse. 6b. El centralizador responde 501 porque el ciudadano ya estaba registrado: el sistema revierte la creación de la carpeta y notifica el conflicto.
Requerimientos asociados	RF-01, RF-02, RF-03, RF-04, RF-05, RF-06, RF-07 · RNF-11, RNF-16, RNF-18
Servicios involucrados	ms-ciudadano (orquesta), ms-interoperabilidad (consume el centralizador), Keycloak (crea la identidad), ms-notificaciones (confirma al ciudadano)

CU-02 · Autenticarse en el operador

Identificador	CU-02
Nombre	Autenticarse en el operador
Actor principal	Ciudadano
Actores secundarios	Proveedor de identidad (Keycloak)
Descripción	El ciudadano acredita su identidad para acceder a su carpeta. La autenticación se delega en un proveedor OIDC y el operador nunca almacena ni procesa contraseñas.
Disparador	El ciudadano intenta acceder a un recurso protegido.
Precondiciones	El ciudadano tiene una carpeta activa y credenciales vigentes.
Postcondiciones	El ciudadano tiene una sesión activa con un token de acceso de vigencia limitada, y la sesión queda registrada en la bitácora de auditoría.
Flujo principal	1. El ciudadano solicita acceso a su carpeta. 2. El sistema lo redirige al proveedor de identidad con el desafío de código. 3. El ciudadano presenta sus credenciales. 4. El proveedor las verifica y exige el segundo factor. 5. El proveedor redirige de vuelta con el código de autorización. 6. El portal canjea el código por los tokens. 7. El gateway valida firma, vigencia y audiencia del token en cada petición posterior.
Flujos alternos y de excepción	3a. Credenciales inválidas: el proveedor niega el acceso y aplica bloqueo progresivo tras intentos fallidos consecutivos. 4a. El segundo factor no se supera: la sesión no se emite. 7a. El token está vencido: el portal lo renueva con el token de refresco sin volver a pedir credenciales. 7b. El token es inválido o fue revocado: el gateway responde 401 y obliga a autenticarse de nuevo.
Requerimientos asociados	RF-37, RF-38, RF-39 · RNF-11, RNF-12, RNF-14
Servicios involucrados	Keycloak (autentica), api-gateway (valida el token), ms-ciudadano (resuelve la carpeta del titular)

CU-03 · Cargar documento a la carpeta

Identificador	CU-03
Nombre	Cargar documento a la carpeta
Actor principal	Ciudadano
Actores secundarios	Almacenamiento de objetos
Descripción	El ciudadano sube un documento a su carpeta junto con los metadatos que permiten clasificarlo. El archivo viaja directamente al almacén de objetos mediante una URL firmada y no atraviesa ningún microservicio.
Disparador	El ciudadano selecciona un archivo y confirma la carga.
Precondiciones	El ciudadano tiene sesión activa y no ha superado su cuota de documentos no certificados.
Postcondiciones	El documento queda almacenado, con sus metadatos registrados y en estado no certificado, y se emite el evento que dispara su autenticación.
Flujo principal	1. El ciudadano indica el título y los metadatos del documento. 2. El sistema lo registra en estado pendiente y solicita una URL firmada de escritura. 3. El sistema devuelve la URL al portal. 4. El portal sube el archivo directamente al almacén. 5. El almacén notifica la creación del objeto. 6. El sistema valida tipo, tamaño y cuota. 7. El sistema publica el evento de documento cargado.
Flujos alternos y de excepción	6a. El tipo o el tamaño no son admitidos: el sistema descarta el objeto y notifica el rechazo. 6b. Se superó la cuota de no certificados: el sistema rechaza la carga e indica cuántos documentos debe liberar. 4a. La URL firmada expiró antes de completar la subida: el portal solicita una nueva sin repetir el registro de metadatos. 5a. El objeto nunca llega: un proceso de limpieza cancela los registros pendientes vencidos.
Requerimientos asociados	RF-12, RF-14, RF-16, RF-17, RF-18 · RNF-02, RNF-08, RNF-10
Servicios involucrados	ms-documentos (registra y valida), almacén de objetos (recibe el archivo), RabbitMQ (propaga el evento)

CU-04 · Autenticar documento ante GovCarpeta

Identificador	CU-04
Nombre	Autenticar documento ante GovCarpeta
Actor principal	GovCarpeta (MinTIC)
Actores secundarios	Ciudadano, almacenamiento de objetos
Descripción	El sistema somete un documento al servicio de autenticación del centralizador y registra el resultado en sus metadatos. La operación se dispara de forma asíncrona a partir del evento de carga.
Disparador	Se recibe el evento de documento cargado, o el ciudadano solicita explícitamente la autenticación.
Precondiciones	El documento existe en el almacén y el operador está registrado ante el centralizador.
Postcondiciones	El documento queda marcado como certificado o como rechazado, y el resultado se registra en la bitácora de auditoría.
Flujo principal	1. El servicio de interoperabilidad recibe el evento de carga. 2. Solicita al almacén una URL firmada de lectura de vigencia corta. 3. Invoca el servicio de autenticación del centralizador con el identificador del ciudadano, la URL y el título. 4. El centralizador descarga el documento usando la URL y lo verifica. 5. El centralizador responde el resultado. 6. El sistema publica el evento de documento autenticado. 7. El servicio documental actualiza el estado y notifica al ciudadano.
Flujos alternos y de excepción	5a. El centralizador responde 204 o 501: el documento queda en estado no certificado y el ciudadano recibe el motivo. 3a. El centralizador no responde: la operación se reintenta con retroceso exponencial y, superado el umbral, el disyuntor se abre y el documento queda en cola de reproceso. 2a. La URL expira antes de que el centralizador la use: se emite una nueva y se reintenta.
Requerimientos asociados	RF-15, RF-22, RF-39 · RNF-13, RNF-19, RNF-22, RNF-25
Servicios involucrados	ms-interoperabilidad (invoca el centralizador), ms-documentos (actualiza el estado), almacén de objetos (emite la URL firmada)

CU-09 · Solicitar traslado a otro operador

Identificador	CU-09
Nombre	Solicitar traslado a otro operador
Actor principal	Ciudadano
Actores secundarios	GovCarpeta, operador destino
Descripción	El ciudadano traslada su carpeta completa a otro operador. La operación es una saga distribuida: implica liberar la afiliación en el centralizador, transferir los documentos directamente al operador destino y esperar su confirmación antes de eliminar los datos locales.
Disparador	El ciudadano solicita el traslado indicando el operador destino.
Precondiciones	El ciudadano está afiliado a este operador y el operador destino tiene registrado su punto de recepción ante el centralizador.
Postcondiciones	El ciudadano queda afiliado al operador destino con todos sus documentos, y este operador ya no conserva sus datos.
Flujo principal	1. El ciudadano elige el operador destino. 2. El sistema abre la saga de transferencia. 3. Consulta el directorio y resuelve el punto de recepción del destino. 4. Emite URL firmadas de lectura de todos los documentos. 5. Libera la afiliación en el centralizador. 6. Envía la carpeta al punto de recepción del destino. 7. El destino descarga los documentos y se registra como nuevo operador. 8. El destino confirma la recepción. 9. El sistema elimina los objetos y purga los datos locales. 10. Notifica al ciudadano.
Flujos alternos y de excepción	6a. El operador destino no responde: vencido el tiempo límite, el sistema compensa y vuelve a registrar al ciudadano en este operador. 8a. La confirmación indica fracaso: el sistema compensa de la misma forma y notifica el motivo. 8b. La confirmación llega duplicada: se descarta por idempotencia sobre el identificador de transferencia. 9a. La eliminación falla: la saga queda en estado de purga pendiente y se reintenta; los documentos nunca se eliminan antes de la confirmación.
Requerimientos asociados	RF-08, RF-09, RF-10, RF-29 · RNF-04, RNF-08, RNF-25
Servicios involucrados	ms-ciudadano (inicia), ms-interoperabilidad (orquesta la saga), almacén de objetos, ms-notificaciones

4. Vista de componentes

4.1 Componentes lógicos

La vista lógica organiza la solución en cuatro capas y ocho componentes de dominio. Un componente lógico es una agrupación de responsabilidades, independiente de la tecnología que luego lo materialice.

Vista de componentes lógicos

Agrupación por capacidad de negocio. Cada componente lógico tiene una única razón de cambio y es candidato a microservicio.



Figura 2. Vista de componentes lógicos.

4.2 Componentes técnicos

La vista técnica asigna a cada componente lógico un artefacto desplegable concreto, con su tecnología y su almacén de datos. La correspondencia no es uno a uno: la gestión de identidad se resuelve con un producto existente en lugar de un servicio propio, y las capas de canal y fachada se materializan en aplicaciones de frontera.

Vista de componentes técnicos

Materialización de cada componente lógico en un artefacto desplegable, con su tecnología y su almacén de datos propio.

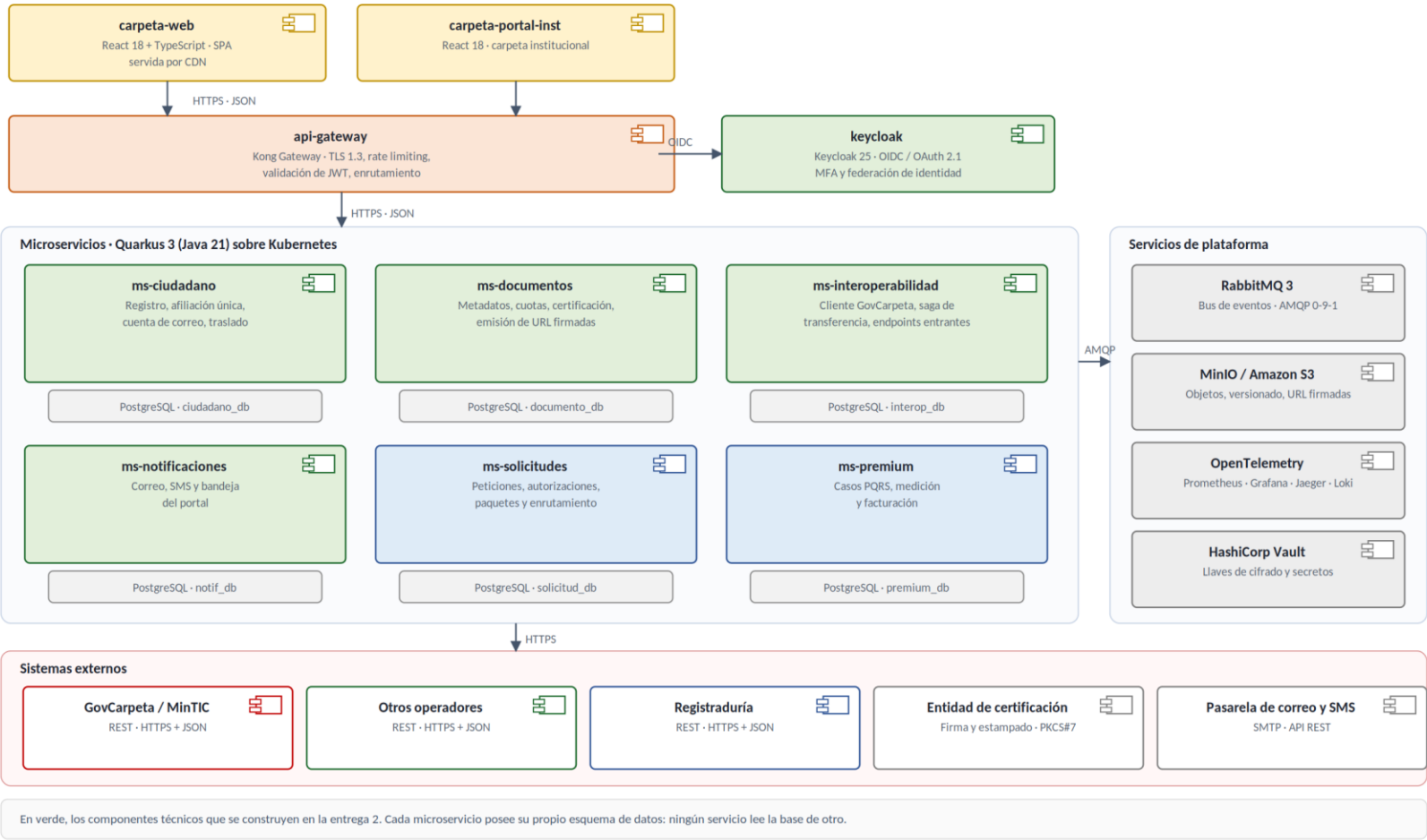


Figura 3. Vista de componentes técnicos.

Componente	Tecnología	Responsabilidad técnica	RNF que atiende
carpeta-web	React 18 + TypeScript, servida por CDN	Portal del ciudadano: registro, carpeta, carga y autorizaciones.	RNF-16, RNF-17
api-gateway	Kong Gateway	Punto único de entrada. Termina TLS, valida el JWT, aplica límites de tasa y enruta.	RNF-03, RNF-10, RNF-11
keycloak	Keycloak 25 (OIDC / OAuth 2.1)	Proveedor de identidad con segundo factor y federación.	RNF-11, RNF-12
ms-ciudadano	Quarkus 3 sobre Java 21	Servicio de dominio del titular y su afiliación.	RNF-05, RNF-24
ms-documentos	Quarkus 3 sobre Java 21	Servicio de dominio documental y emisor de URL firmadas.	RNF-02, RNF-06, RNF-08
ms-interoperabilidad	Quarkus 3 sobre Java 21	Cliente de GovCarpeta, orquestador de la saga y receptor de transferencias.	RNF-18, RNF-19, RNF-22, RNF-25
ms-notificaciones	Quarkus 3 sobre Java 21	Consumidor de eventos y emisor de avisos.	RNF-16, RNF-25
PostgreSQL	Instancia gestionada, una base por servicio	Persistencia transaccional aislada por servicio.	RNF-09, RNF-24
MinIO / Amazon S3	Almacenamiento de objetos compatible con S3	Documentos, versionado y URL firmadas.	RNF-08, RNF-10, RNF-21
RabbitMQ	AMQP 0-9-1	Bus de eventos que desacopla los servicios en el tiempo.	RNF-04, RNF-25
OpenTelemetry + Prometheus + Grafana + Jaeger + Loki	Instrumentación y telemetría	Métricas, trazas distribuidas y registros centralizados.	RNF-23
HashiCorp Vault	Gestión de secretos	Llaves de cifrado y credenciales fuera del código.	RNF-10, RNF-15

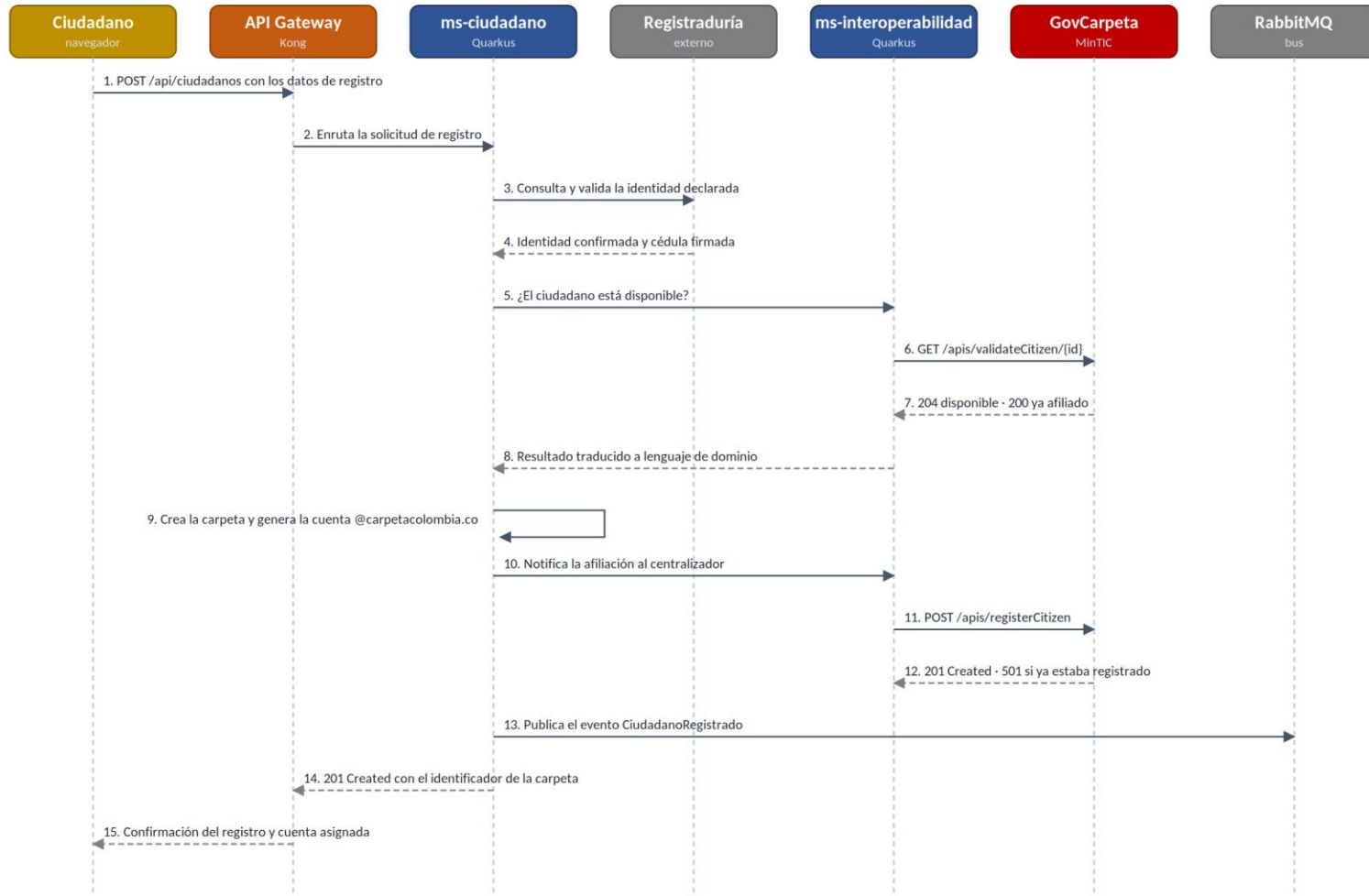
5. Diagramas de secuencia

Se documentan cuatro operaciones. Las tres primeras corresponden a los casos de uso que se implementan en esta entrega; la cuarta es el traslado entre operadores, que se incluye por ser la operación con mayor riesgo de la solución.

5.1 CU-01 Registro de un ciudadano

Diagrama de secuencia • CU-01 Registro de un ciudadano

Incluye la validación de afiliación única contra el centralizador, exigida por el RF-03



Decisión de diseño

El paso 5 se delega en ms-interoperabilidad y no se llama a GovCarpeta desde ms-ciudadano. Esa indirección es la capa anticorrupción: aísla el dominio de los códigos de estado no estándar del centralizador y concentra en un solo punto la interpretación de 200 frente a 204. La carga del documento de identidad firmado por la Registraduría se dispara de forma asíncrona a partir del evento del paso 13, para no alargar la transacción de registro.

Figura 4. Secuencia del registro de un ciudadano.

5.2 CU-02 Autenticación del ciudadano

Diagrama de secuencia · CU-02 Autenticación del ciudadano

Flujo OIDC Authorization Code con PKCE. El operador no almacena ni maneja contraseñas.

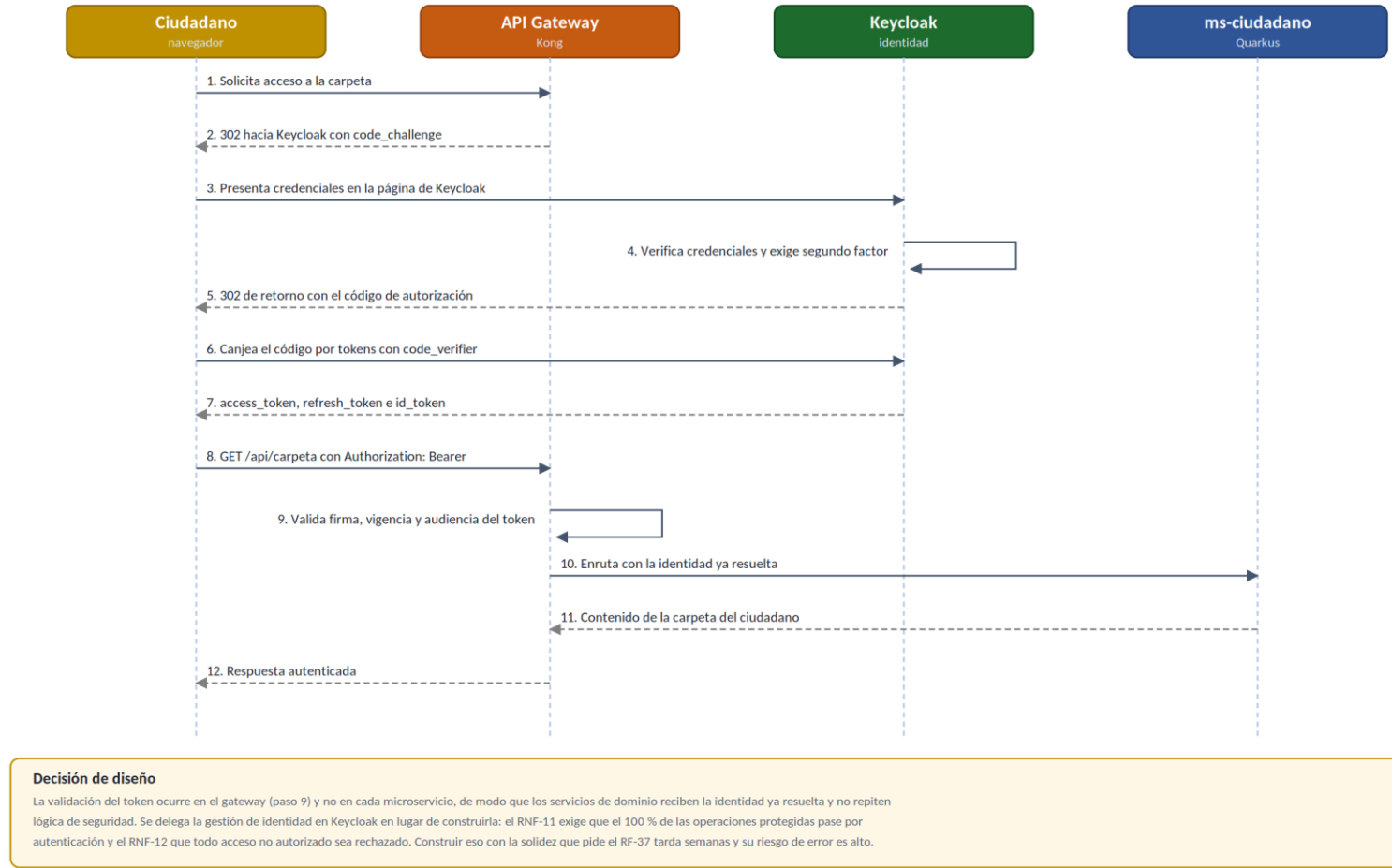
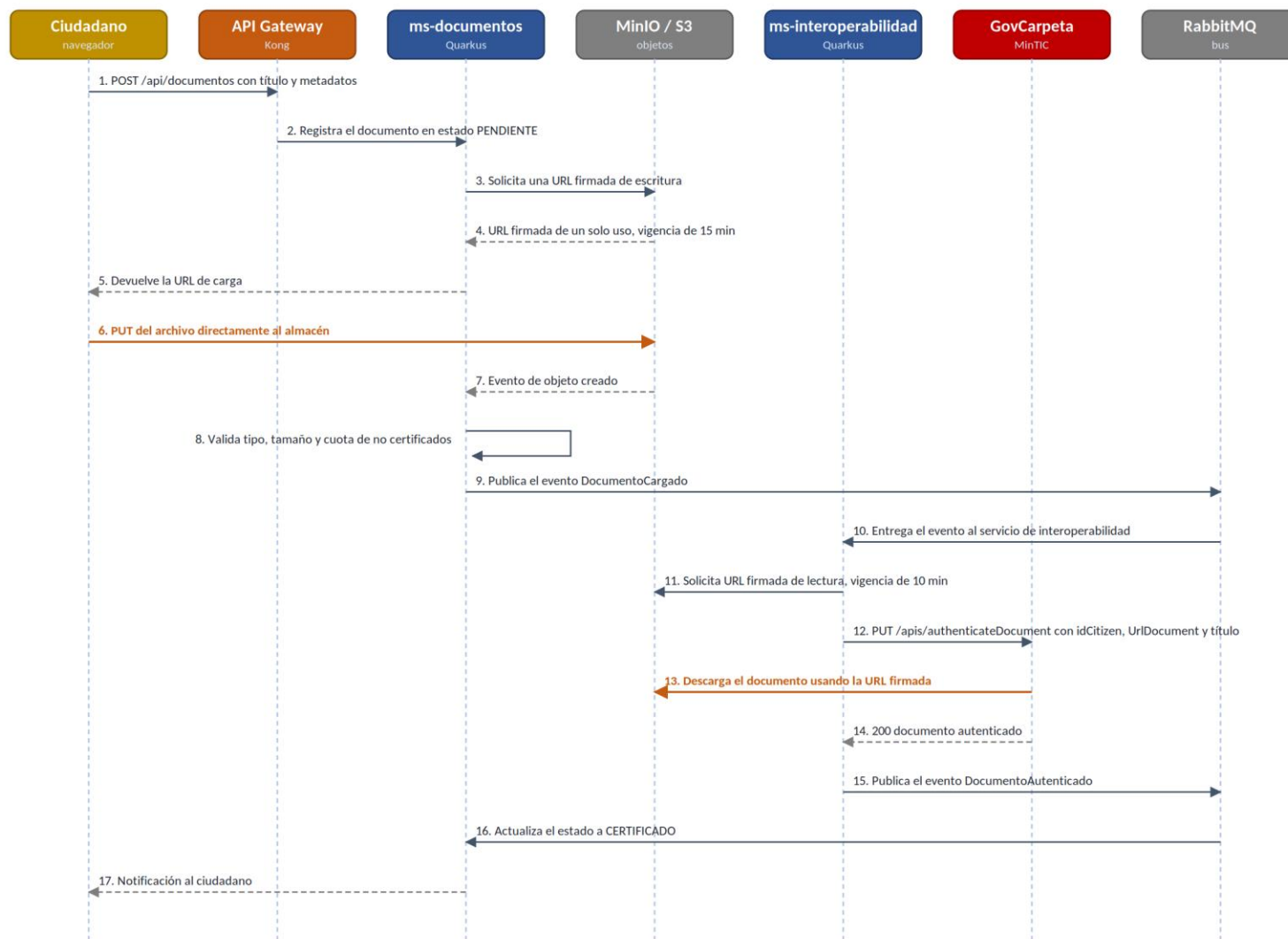


Figura 5. Secuencia de la autenticación del ciudadano mediante OIDC.

5.3 CU-03 y CU-04 Carga y autenticación de documento

Diagrama de secuencia • CU-03 Carga de documento y CU-04 Autenticación ante GovCarpeta

La carga usa subida directa al almacén de objetos; el servicio nunca actúa de intermediario del archivo.



Decisión de diseño

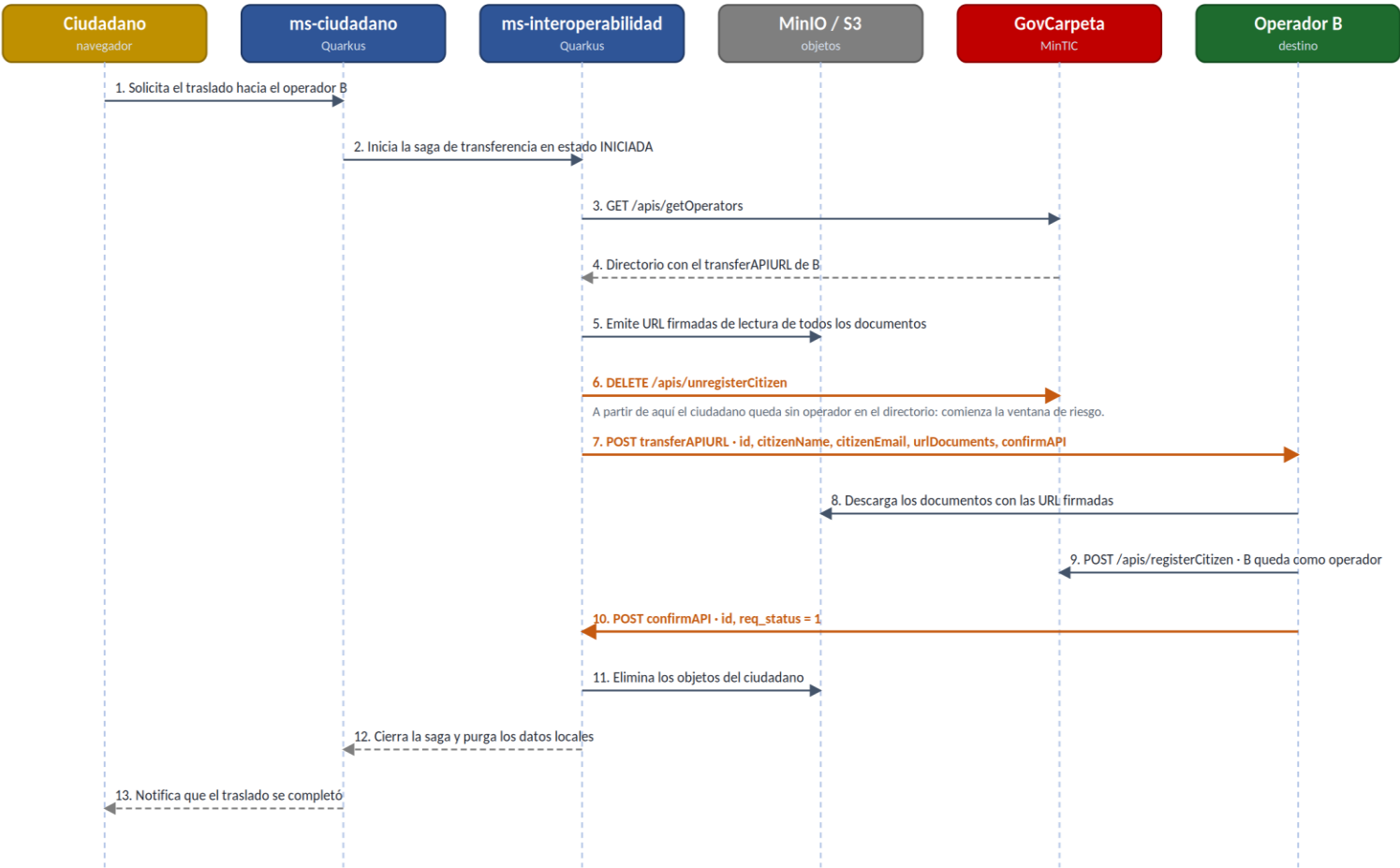
Los pasos 6 y 13 son los únicos en los que viaja el archivo, y en ninguno pasa por un microservicio: el ciudadano escribe directo en el almacén y el centralizador lee directo de él. Eso mantiene los servicios sin estado y evita que el ancho de banda de los documentos consuma memoria de los pods. La contrapartida es que la URL firmada es un portador de acceso: se emite de un solo uso, con vigencia corta, restringida a la lectura y registrada en la bitácora de auditoría que exige el RNF-14.

Figura 6. Secuencia de la carga de un documento y su autenticación ante GovCarpeta.

5.4 CU-09 Traslado a otro operador

Diagrama de secuencia • CU-09 Traslado de un ciudadano a otro operador

Implementa el contrato transferCitizen / transferCitizenConfirm acordado entre los equipos del curso



Compensación y ventana de riesgo

Entre los pasos 6 y 9 el ciudadano no tiene operador asignado en el directorio del centralizador. Si el paso 7, 8 o 10 falla, ese estado inválido se vuelve permanente. El orden viene impuesto por el centralizador, que rechaza registerCitizen con 501 si el ciudadano sigue afiliado, de modo que la liberación debe ocurrir antes. La solución implementa tres protecciones: temporizador sobre la confirmación, compensación automática que vuelve a registrar al ciudadano en A si vence o si llega req_status = 0, y reintentos idempotentes en los pasos 6 y 9. Los documentos solo se eliminan en el paso 11, nunca antes de la confirmación.

Figura 7. Secuencia del traslado de un ciudadano a otro operador.

6. Contrato de interoperabilidad entre operadores

6.1 Acuerdo adoptado

El centralizador registra y publica la dirección del punto de recepción de cada operador, pero en ninguna parte define qué estructura debe tener el mensaje que un operador envía a otro. Ese vacío, señalado en el capítulo 8.4 de la entrega anterior como el riesgo de mayor impacto del proyecto, se cerró mediante un acuerdo entre los equipos del curso, que esta solución adopta.

El acuerdo consta de dos operaciones y de una regla de secuencia. La regla establece que el operador origen libera primero la afiliación en el centralizador, envía después la carpeta al destino y solo elimina sus datos cuando recibe la confirmación. Ese orden no es una preferencia de diseño: viene impuesto por el centralizador, que rechaza el registro del destino mientras el ciudadano siga afiliado al origen.

Operación	Propósito	Cuerpo del mensaje	Invocación
transferCitizen	Recibe el traslado de un ciudadano desde otro operador.	id, citizenName, citizenEmail, urlDocuments, confirmAPI	POST sobre el transferAPIURL publicado por el operador destino.
transferCitizenConfirm	Informa al operador origen el resultado de la recepción.	id, req_status (1 éxito, 0 fracaso)	POST sobre la URL que el origen envió en confirmAPI.

6.2 Brechas identificadas en el acuerdo

El acuerdo resuelve el problema principal, que era la ausencia de contrato. Quedan sin embargo seis brechas que conviene cerrar antes de que los equipos comiencen a implementar, porque todas producen fallas difíciles de diagnosticar una vez el código está escrito. Se documentan aquí como aporte de este equipo a la discusión.

ID	Brecha	Descripción	Propuesta
B-01	El mensaje no transporta metadatos	urlDocuments solo lleva la ubicación del archivo. Los metadatos que clasifican el documento, identifican la entidad que lo avala y fijan sus fechas se pierden en el traslado.	Agregar por cada documento su título, entidad emisora, fecha de expedición y estado de certificación. Sin esto se incumple el RF-14 y el documento pierde valor probatorio al cambiar de operador.
B-02	No se identifica al operador origen	El destino no puede verificar quién le envía la carpeta ni validar que el envío sea legítimo.	Incluir el operatorId del origen y firmar la petición o autenticarla con un token compartido.
B-03	No existe identificador de transferencia	Si la confirmación se pierde y se reintenta, el destino no puede distinguir un reintento de una transferencia nueva.	Agregar un transferId único y tratar la operación como idempotente sobre esa clave.
B-04	req_status no explica el fracaso	Un valor de 0 no permite distinguir un error transitorio, que merece reintento, de uno definitivo, que exige compensación.	Agregar un campo de motivo. Con un solo bit, el origen no puede decidir si reintentar o compensar.
B-05	No se acordó la vigencia de las URL	Si expiran antes de que el destino termine de descargar, la transferencia falla sin causa aparente.	Fijar una vigencia mínima acordada y permitir que el destino solicite su renovación.
B-06	No hay tiempo límite de confirmación	El flujo propuesto espera la confirmación sin plazo. Si nunca llega, el ciudadano queda sin operador de forma indefinida.	Definir un tiempo límite acordado entre los equipos tras el cual el origen compensa y vuelve a registrarse.

La más grave es B-01. Un documento que llega al operador destino sin la entidad que lo avala ni su fecha de expedición deja de ser un documento certificado y pasa a ser un archivo suelto. El caso de estudio es explícito en que los metadatos permiten determinar qué entidad avala el documento y qué fechas tiene asociadas; perderlos en el traslado vacía de contenido la promesa de conservación con valor probatorio.

7. Diagrama de despliegue

7.1 Diagrama

El despliegue se resuelve en su totalidad sobre nube pública gestionada. El diagrama muestra los nodos de ejecución, los artefactos que alojan y las tecnologías asociadas.

Diagrama de despliegue

Nodos, tecnologías, protocolos y formatos de mensajería de la solución en producción.

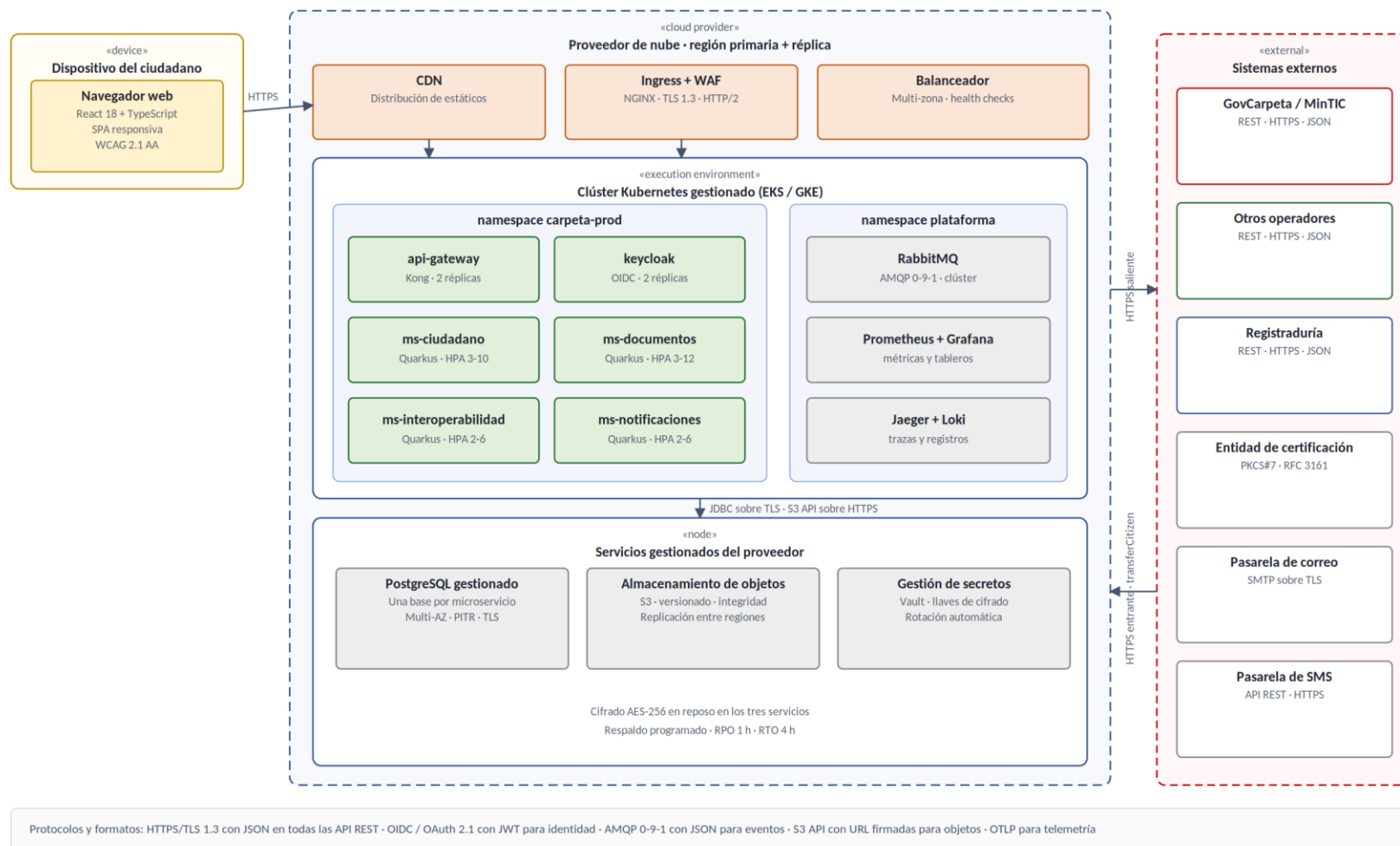


Figura 8. Diagrama de despliegue de la solución en producción.

7.2 Protocolos y formatos de mensajería

La tabla detalla, para cada conexión del diagrama anterior, el protocolo de transporte y el formato del mensaje.

Conexión	Protocolo	Formato de mensaje	Observación
Navegador y API Gateway	HTTPS sobre TLS 1.3, HTTP/2	JSON (application/json)	Único canal expuesto a Internet. Certificados gestionados y rotación automática.
Navegador y almacén de objetos	HTTPS con URL firmada	Binario (multipart)	Subida y descarga directas. El archivo nunca atraviesa un microservicio.
Gateway y microservicios	HTTP/2 dentro del clúster	JSON	Tráfico interno. mTLS opcional mediante malla de servicios en fase posterior.
Microservicio y bus de eventos	AMQP 0-9-1	JSON con esquema versionado	Entrega garantizada con confirmación y cola de mensajes fallidos.
Microservicio y base de datos	JDBC sobre TLS	Protocolo nativo de PostgreSQL	Una base por servicio. Ningún servicio consulta la base de otro.
Solución y GovCarpeta	HTTPS sobre TLS 1.2 o superior	JSON	Siete operaciones REST. Reintento con retroceso exponencial y disyuntor.
Solución y otros operadores	HTTPS sobre TLS 1.2 o superior	JSON (contrato acordado entre equipos)	transferCitizen entrante y saliente, y transferCitizenConfirm.
Identidad	OIDC Authorization Code con PKCE	JWT firmado con RS256	Validación de firma y vigencia en el gateway.
Telemetría	OTLP sobre gRPC	Protobuf	Trazas, métricas y registros a la plataforma de observabilidad.

8. Decisiones de arquitectura

Cada decisión se documenta siguiendo la plantilla de decisión arquitectural del método UAM: problema, contexto, alcance, restricciones, supuestos, alternativas evaluadas, racional e implicaciones. La última fila de cada ficha indica el requerimiento no funcional que la decisión atiende, de modo que ninguna decisión quede sin justificación medible.

Conviene señalar algo sobre la naturaleza de estas decisiones. Ninguna es gratuita: todas cuestan algo. Documentar lo que se pierde importa tanto como documentar lo que se gana, porque es lo que permitirá revisarlas cuando el contexto cambie.

8.1 AD-01 Estilo arquitectural

Decisión	Adoptar una arquitectura de microservicios descompuesta por capacidad de negocio, con base de datos por servicio.
Problema	¿Qué estilo arquitectural permite cumplir simultáneamente la escala nacional, la disponibilidad casi total y el despliegue independiente que exige la especificación?
Contexto	La solución debe poder crecer en capacidad a medida que aumenta el número de ciudadanos, con perfiles de carga muy distintos por capacidad: la consulta de documentos es intensiva en lectura, la interoperabilidad depende de sistemas externos lentos y poco fiables, y la analítica es un trabajo por lotes.
Alcance	Aplica a toda la solución del operador. No aplica al centralizador ni a los sistemas externos.
Restricciones	El equipo es pequeño y el plazo es el de un semestre. La complejidad operativa de los microservicios es real y debe poder gestionarse con las herramientas del curso.
Supuestos	Se dispone de contenedores y orquestación, que son prerrequisito no negociable de este estilo.
Alternativas evaluadas	1. Monolito modular: mucho más simple y barato, transacciones locales, sin latencia de red; pero el despliegue es todo o nada y el escalado obliga a replicar el bloque completo, incluidos los componentes que no lo necesitan. 2. Microservicios por capacidad de negocio: despliegue y escalado independientes, aislamiento de fallos; a cambio de consistencia eventual, complejidad operativa y observabilidad obligatoria. 3. Microservicios por capa técnica: descartada de entrada, produce un monolito distribuido.
Racional	El factor decisivo es el aislamiento de fallos frente a dependencias externas poco fiables. En un monolito, la caída del centralizador o la lentitud de la Registraduría agotan el mismo grupo de hilos que atiende la consulta de la carpeta, y un problema en una integración periférica degrada la función central del sistema. La separación en servicios con disyuntor y mamparo contiene ese efecto. A esto se suma que las capacidades tienen perfiles de escalado opuestos, y replicar el bloque completo para atender picos de lectura desperdicia recursos.
Impactos e implicaciones	Positivo: escalado selectivo, despliegue independiente, aislamiento de fallos, límites de equipo claros. Negativo: se renuncia a las transacciones ACID entre capacidades y aparecen sagas con compensación, como la del traslado; se vuelven obligatorias la observabilidad distribuida y una canalización de despliegue madura; las pruebas de integración se encarecen. Mitigación: comenzar con seis servicios y no fragmentar más hasta que exista una razón de cambio que lo justifique.
RNF que atiende	RNF-01, RNF-05, RNF-24, RNF-25

8.2 AD-02 Modelo de despliegue

Decisión	Desplegar la totalidad de la solución en nube pública gestionada, sin componentes en infraestructura propia.
Problema	¿Dónde se ejecutan los componentes: en nube pública, en infraestructura propia o en un modelo híbrido?
Contexto	El enunciado establece que no hay restricciones sobre el lugar de almacenamiento y admite expresamente nube fuera del país. Los operadores son actores privados que aportan su propia infraestructura y asumen el costo de administración.
Alcance	Aplica a cómputo, persistencia, almacenamiento de objetos y plataforma de observabilidad.
Restricciones	Los documentos certificados deben recuperarse sin alteración durante toda su permanencia en el sistema, y los puntos de recepción de transferencias deben ser alcanzables desde Internet.
Supuestos	El régimen de transferencia internacional de datos personales se cumple mediante cláusulas contractuales con el proveedor.
Alternativas evaluadas	1. Infraestructura propia: control total y ausencia de dependencia de un proveedor; exige inversión inicial alta, un equipo de operación permanente y obliga a construir y operar la redundancia que la integridad documental necesita. 2. Nube pública gestionada: elasticidad real, durabilidad y replicación geográfica incluidas, costo variable; a cambio de dependencia del proveedor y de un costo que crece con el uso. 3. Híbrido con documentos en el país: respondería a una exigencia de residencia de datos que el caso no impone, y duplicaría la complejidad operativa.
Racional	Dos exigencias inclinan la decisión de forma clara. La primera es la integridad documental: conservar sin alteración documentos que permanecen de forma indefinida exige versionado, verificación de integridad y replicación, que el almacenamiento de objetos gestionado ofrece como característica estándar y que en infraestructura propia habría que construir y sostener. La segunda es la capacidad de crecer: convertir recursos adicionales en throughput, como exige el RNF-05, es inmediato en nube y depende de un ciclo de compra de hardware en infraestructura propia. El híbrido se descarta porque el enunciado no impone residencia de datos, y adoptarlo sin necesidad solo agregaría complejidad.
Impactos e implicaciones	Positivo: sin inversión inicial, redundancia y replicación incluidas, capacidad ajustable bajo demanda, menor carga operativa. Negativo: costo operativo recurrente que crece con el volumen almacenado, dependencia del proveedor, y obligación de cumplir el régimen de transferencia internacional. Mitigación: empaquetar todo en contenedores y evitar servicios propietarios sin equivalente abierto, de modo que la migración sea posible.
RNF que atiende	RNF-01, RNF-08, RNF-09, RNF-20, RNF-21

8.3 AD-03 Plataforma de ejecución

Decisión	Ejecutar los servicios en contenedores conformes a OCI orquestados por Kubernetes gestionado.
Problema	¿Qué plataforma ejecuta y opera los microservicios en producción?
Contexto	Seis servicios con réplicas variables, que deben sobrevivir a la caída de un nodo, escalar por métricas y actualizarse sin interrupción del servicio.
Alcance	Aplica a todos los servicios propios y a los productos desplegados en el clúster.
Restricciones	El equipo debe poder reproducir el entorno localmente para desarrollar y probar.
Supuestos	Se dispone de Minikube en las máquinas del equipo, lo que garantiza paridad entre desarrollo y producción.
Alternativas evaluadas	1. Máquinas virtuales con despliegue manual: simple de entender, pero sin recuperación automática ni escalado por métricas, y con actualizaciones que implican interrupción. 2. Plataforma sin servidor: elimina la gestión de infraestructura y cobra por uso, pero el arranque en frío penaliza la latencia y el modelo encaja mal con servicios de larga vida que consumen de una cola. 3. Kubernetes gestionado: recuperación automática, escalado horizontal, actualizaciones progresivas y reversión inmediata, a cambio de una curva de aprendizaje pronunciada.
Racional	Kubernetes aporta directamente cuatro capacidades que de otro modo habría que construir: recuperación automática ante fallo de un pod o de un nodo, escalado horizontal disparado por métricas, actualización progresiva sin interrupción y descubrimiento de servicios por DNS interno. El modelo declarativo, además, hace que el estado deseado del sistema sea un artefacto versionado en el repositorio y no una serie de pasos manuales irreproducibles.
Impactos e implicaciones	Positivo: alta disponibilidad y escalado automático sin desarrollo propio, paridad entre desarrollo y producción, base para despliegues canario y azul-verde. Negativo: curva de aprendizaje alta y una capa más que diagnosticar cuando algo falla; el clúster gestionado tiene un costo base aunque la carga sea baja. Mitigación: usar Minikube en desarrollo y plantillas de Helm comunes a todos los servicios.
RNF que atiende	RNF-01, RNF-05, RNF-20, RNF-24, RNF-25

8.4 AD-04 Lenguaje y marco de trabajo

Decisión	Construir los microservicios en Java 21 con Quarkus 3 y las especificaciones de MicroProfile.
Problema	¿Con qué lenguaje y marco de trabajo se construyen los servicios de dominio?
Contexto	Seis servicios que deben arrancar rápido para escalar bajo demanda, consumir poca memoria para maximizar la densidad por nodo, y exponer sin esfuerzo adicional la instrumentación que exige la observabilidad.
Alcance	Aplica a los servicios propios. No aplica al portal ni a los productos de plataforma.
Restricciones	El equipo debe poder ser productivo dentro del semestre. La bibliografía del curso está construida sobre este ecosistema.
Supuestos	Los integrantes tienen formación previa en Java.
Alternativas evaluadas	1. Spring Boot: el ecosistema más extendido y con mayor oferta de documentación y ejemplos; a cambio de un arranque más lento y un consumo de memoria mayor por instancia. 2. Quarkus con MicroProfile: arranque en decenas de milisegundos, consumo de memoria sensiblemente menor, compilación nativa opcional e implementaciones de tolerancia a fallos y telemetría incluidas en el estándar. 3. Node.js o Python: menor curva inicial y desarrollo rápido, pero un ecosistema menos maduro para transacciones y patrones de resiliencia empresariales.
Racional	La combinación de arranque rápido y bajo consumo de memoria no es una preferencia estética: es lo que permite añadir instancias con rapidez y convertir recursos adicionales en throughput, que es exactamente lo que el RNF-05 exige poder medir. Un servicio que tarda decenas de segundos en estar listo obliga a mantener capacidad ociosa de reserva. A esto se suma que MicroProfile incorpora disyuntor, reintento, mamparo y telemetría como parte de la especificación, sin bibliotecas adicionales, que es exactamente lo que la arquitectura necesita frente a dependencias externas poco fiables. Finalmente, el material de referencia del curso está construido sobre este ecosistema, lo que reduce el riesgo del equipo.
Impactos e implicaciones	Positivo: mayor densidad de servicios por nodo, escalado más ágil, resiliencia y observabilidad disponibles por estándar, alineación con la bibliografía del curso. Negativo: comunidad más pequeña que la de la alternativa dominante y menos respuestas disponibles ante problemas poco comunes. Mitigación: limitarse a las extensiones oficiales y no adoptar compilación nativa hasta que el servicio esté estabilizado.
RNF que atiende	RNF-03, RNF-05, RNF-23, RNF-25

8.5 AD-05 Persistencia y almacenamiento documental

Decisión	Separar la persistencia en dos planos: PostgreSQL gestionado, con una base por servicio, para los datos estructurados; y almacenamiento de objetos compatible con S3, con acceso mediante URL firmadas, para los documentos.
Problema	¿Dónde y cómo se almacenan los datos estructurados y los documentos, que tienen requerimientos completamente distintos?
Contexto	Los metadatos son relacionales, se consultan y filtran constantemente y exigen integridad transaccional. Los documentos son binarios inmutables, potencialmente grandes, que se escriben una vez y se leen ocasionalmente, y deben conservarse a perpetuidad.
Alcance	Aplica a todos los servicios y al ciclo de vida completo del documento.
Restricciones	El servicio de autenticación del centralizador no recibe el archivo: recibe una URL con la que lo descarga, de modo que el almacén debe poder emitir URL temporales firmadas.
Supuestos	El almacenamiento de objetos del proveedor ofrece versionado, cifrado en reposo y replicación entre regiones.
Alternativas evaluadas	1. Documentos como campos binarios en la base relacional: simplifica el respaldo al tener una sola fuente, pero degrada el rendimiento de la base, encarece el almacenamiento y hace inviable la emisión de URL firmadas. 2. Base compartida entre servicios: ahorra infraestructura, pero reintroduce el acoplamiento de esquema que la decisión AD-01 pretendía eliminar. 3. Plano estructurado y plano de objetos separados, con una base por servicio: aísla los servicios y permite que el archivo nunca atraviese la aplicación.
Racional	Mantener los documentos fuera de la base y fuera del camino de la aplicación resuelve tres problemas a la vez. Primero, hace posible cumplir el contrato del centralizador, que exige una URL descargable. Segundo, mantiene los servicios sin estado y con un consumo de memoria predecible, porque el ancho de banda de los archivos no pasa por los pods. Tercero, traslada al proveedor la redundancia y la verificación de integridad que exige el RNF-08, que ofrece como característica estándar. La base por servicio, por su parte, es la condición que hace real la independencia de despliegue: sin ella, un cambio de esquema se propaga a servicios que no lo esperaban.
Impactos e implicaciones	Positivo: servicios sin estado y escalables, integridad y redundancia delegadas, cumplimiento del contrato externo, aislamiento entre servicios. Negativo: no hay transacciones entre el plano de objetos y el relacional, de modo que la consistencia entre el archivo y su metadato debe garantizarse con eventos y procesos de conciliación; las URL firmadas son portadores de acceso y exigen vigencia corta y auditoría. Mitigación: patrón de bandeja de salida para no perder eventos, y proceso periódico de conciliación entre objetos y metadatos.
RNF que atiende	RNF-02, RNF-06, RNF-08, RNF-09, RNF-10, RNF-24

8.6 AD-06 Comunicación entre servicios

Decisión	Usar comunicación síncrona sobre HTTP solo cuando el usuario espera la respuesta, y mensajería asíncrona sobre RabbitMQ para todo lo demás.
Problema	¿Cómo se comunican los microservicios entre sí y con los sistemas externos?
Contexto	Algunas operaciones son interactivas y el ciudadano espera frente a la pantalla. Otras son de fondo y dependen de sistemas externos lentos y poco fiables, como el centralizador o la pasarela de notificaciones.
Alcance	Aplica a la comunicación entre servicios propios y con sistemas externos.
Restricciones	El enunciado descarta el tiempo real, pero exige que la latencia sea tan baja como sea posible.
Supuestos	Las operaciones asíncronas toleran consistencia eventual del orden de segundos.
Alternativas evaluadas	1. Todo síncrono sobre HTTP: es el modelo más simple de entender y depurar, pero acopla los servicios en el tiempo y propaga los fallos en cascada. 2. Todo asíncrono sobre mensajería: máximo desacople y resiliencia, pero complica innecesariamente las operaciones interactivas y degrada la experiencia del usuario. 3. Modelo mixto según si el usuario espera la respuesta.
Racional	El criterio de separación es concreto y verificable: si el ciudadano está esperando frente a la pantalla, la llamada es síncrona; si no, es un evento. La autenticación de un documento ante el centralizador ilustra el punto. Hacerla síncrona dentro de la carga significaría que la lentitud o la caída del centralizador deja al ciudadano esperando o le impide subir documentos, cuando en realidad la subida ya tuvo éxito. Al convertirla en un evento, la carga responde de inmediato y la autenticación ocurre después, con reintentos, sin que el usuario perciba el problema.
Impactos e implicaciones	Positivo: desacople temporal, tolerancia a la caída de dependencias externas, absorción de picos de carga. Negativo: consistencia eventual visible para el usuario, que debe reflejarse en la interfaz mediante estados intermedios explícitos; el bus se vuelve infraestructura crítica y hay que operarlo con alta disponibilidad y colas de mensajes fallidos. Mitigación: bandeja de salida transaccional para no perder eventos, consumidores idempotentes y cola de fallidos con alerta.
RNF que atiende	RNF-03, RNF-04, RNF-07, RNF-25

8.7 AD-07 Gestión de identidad

Decisión	Delegar la autenticación en Keycloak mediante OIDC con Authorization Code y PKCE, y validar el token en el gateway.
Problema	¿Se construye la gestión de identidad o se adopta un producto existente?
Contexto	La especificación exige que ninguna operación protegida se ejecute sin autenticación previa y que todo intento de acceso no autorizado sea rechazado, con control de acceso en función de la identidad del usuario, el documento solicitado y las autorizaciones concedidas por su titular.
Alcance	Aplica a ciudadanos y a usuarios institucionales.
Restricciones	El operador custodia documentos con valor probatorio. Un fallo de autenticación compromete datos personales sensibles y tiene consecuencias legales.
Supuestos	El proveedor de identidad puede desplegarse en el mismo clúster y federarse en el futuro con sistemas de identidad del Estado.
Alternativas evaluadas	1. Autenticación propia con usuario y contraseña: control total y sin dependencias; pero implica implementar almacenamiento seguro de credenciales, segundo factor, recuperación de cuenta, bloqueo progresivo y rotación de tokens, todo ello con alto riesgo de error. 2. Keycloak autoalojado: implementación probada de OIDC con segundo factor, federación y gestión de sesiones, a cambio de operar un componente más. 3. Servicio de identidad del proveedor de nube: menor carga operativa pero mayor dependencia y menor portabilidad.
Racional	La identidad es el área donde un error propio es más costoso y menos perdonable. Implementar correctamente el almacenamiento de credenciales, el segundo factor, la rotación de tokens y el bloqueo progresivo lleva semanas y produce un resultado inferior al de un producto maduro. Adoptar Keycloak elimina ese trabajo y ese riesgo, y de paso hace que el operador nunca custodie contraseñas, lo que reduce la superficie de ataque. Validar el token en el gateway en lugar de en cada servicio evita duplicar lógica de seguridad y permite que los servicios de dominio reciban la identidad ya resuelta.
Impactos e implicaciones	Positivo: segundo factor y federación disponibles de inmediato, ausencia de custodia de contraseñas, menor superficie de ataque. Negativo: un componente más que operar y mantener actualizado, y que se convierte en punto único de fallo para el acceso. Mitigación: desplegarlo con al menos dos réplicas y respaldo de su almacén; mantener las sesiones vigentes durante una caída para no expulsar a los usuarios ya autenticados.
RNF que atiende	RNF-11, RNF-12, RNF-14, RNF-15

8.8 AD-08 Transferencia entre operadores

Decisión	Implementar el traslado como una saga con compensación, temporizador sobre la confirmación y operaciones idempotentes.
Problema	¿Cómo se garantiza la consistencia del traslado si la liberación y el nuevo registro son dos llamadas independientes a un sistema de terceros?
Contexto	El centralizador rechaza el registro del operador destino mientras el ciudadano siga afiliado al origen, de modo que la liberación debe ocurrir antes. Entre la liberación y el nuevo registro existe una ventana en la que el ciudadano no tiene operador asignado.
Alcance	Aplica al traslado saliente y a la recepción de traslados entrantes.
Restricciones	El orden de las operaciones viene impuesto por el centralizador y no es negociable. El acuerdo entre equipos define el contrato pero no fija tiempos límite ni identificadores de transferencia.
Supuestos	El operador destino implementa el contrato acordado y confirma la recepción.
Alternativas evaluadas	1. Transacción distribuida con confirmación en dos fases: garantizaría atomicidad, pero es inviable porque exigiría que el centralizador y el operador par participaran del protocolo, y ninguno lo soporta. 2. Envío optimista sin confirmación: simple, pero cualquier fallo del destino produce pérdida de documentos sin que el origen se entere. 3. Saga con compensación y confirmación explícita.
Racional	No hay forma de hacer atómica una operación repartida entre tres sistemas independientes que no comparten protocolo transaccional. Lo que sí puede garantizarse es que ningún estado inválido sea permanente, y eso es exactamente lo que aporta la saga: si la confirmación no llega dentro del plazo o llega con resultado negativo, el origen vuelve a registrarse como operador del ciudadano y lo deja en el estado anterior. La idempotencia sobre el identificador de transferencia es lo que permite reintentar sin duplicar, y la regla de no eliminar documentos antes de la confirmación es la que garantiza que nunca se pierda información.
Impactos e implicaciones	Positivo: ningún estado inválido queda permanente, los documentos nunca se eliminan antes de confirmar, la operación es reintenable. Negativo: hay que mantener una máquina de estados persistente por transferencia y un proceso que vigile los plazos vencidos; el ciudadano puede quedar unos minutos sin operador visible en el directorio. Mitigación: cerrar con los demás equipos las brechas B-03, B-04 y B-06 del capítulo 6, que son precisamente las que esta decisión necesita para funcionar.
RNF que atiende	RNF-08, RNF-09, RNF-22, RNF-25

9. Alcance de la implementación

La entrega implementa cuatro casos de uso de extremo a extremo. La tabla precisa el punto de entrada de cada uno y los componentes que intervienen.

Caso de uso	Operación	Punto de entrada	Componentes involucrados
CU-01	Registro de un ciudadano	POST /api/ciudadanos	ms-ciudadano, ms-interoperabilidad, Keycloak. Consume validateCitizen y registerCitizen.
CU-02	Autenticación del ciudadano	Flujo OIDC sobre Keycloak	Keycloak y api-gateway. El operador no custodia contraseñas.
CU-03	Carga de documento	POST /api/documentos y PUT directo al almacén	ms-documentos y almacén de objetos. Emisión de URL firmada de escritura.
CU-04	Autenticación de documento	Disparado por el evento DocumentoCargado	ms-interoperabilidad. Consume authenticateDocument con URL firmada de lectura.

Los cuatro casos no se eligieron por conveniencia: forman la cadena mínima que demuestra que la arquitectura funciona. El registro ejercita la integración con el centralizador y la validación de afiliación única; la autenticación ejercita la delegación de identidad; la carga ejercita el almacenamiento de objetos y la emisión de URL firmadas; y la autenticación de documento ejercita la comunicación asíncrona y la capa anticorrupción. Entre los cuatro tocan todos los componentes que se despliegan en esta entrega.

10. Trazabilidad con la entrega 1

La matriz verifica que ningún elemento de esta arquitectura nace sin respaldo en la especificación anterior, y que ningún atributo de calidad quedó sin decisión que lo atienda.

Artefacto de la entrega 2	Contenido	Origen en la entrega 1	Observación
Casos de uso	CU-01 a CU-12	RF-01 a RF-44 de la entrega 1	Todo caso de uso nace de al menos un requerimiento funcional ya especificado.
Componentes lógicos	Ocho componentes de dominio	Las ocho capacidades del capítulo 3 de la entrega 1	La descomposición conserva los límites definidos en la especificación.
Componentes técnicos	Seis microservicios más Keycloak	RNF-05, RNF-24, RNF-25	El despliegue acotado al servicio modificado es lo que hace verificable el RNF-24.
Despliegue	Nube gestionada con Kubernetes	RNF-01, RNF-08, RNF-09, RNF-21	La redundancia y la capacidad de crecer bajo demanda no son alcanzables con infraestructura propia al alcance del equipo.
Decisiones	AD-01 a AD-08	Mapeo de QoS del capítulo 5 de la entrega 1	Cada decisión referencia el atributo de calidad que la justifica.

Artefacto de la entrega 2	Contenido	Origen en la entrega 1	Observación
Interoperabilidad	Contrato entre operadores	RF-29 y capítulo 8.4 de la entrega 1	El riesgo principal que la entrega 1 dejó abierto queda cerrado por el acuerdo entre equipos, con las seis brechas anotadas.